Two-Minute Interview Narration

A concise explanation of the movie recommendation pipeline.

Project: Movie Recommender System

Source archive supplied in this conversation: Movie-recommender-system-main.zip | Notebook: notebook.ipynb | App: app.py

Narration (approximately two minutes)

“I built a content-based movie recommender using the TMDB 5000 movies and credits datasets. The goal was to recommend five movies similar to a title selected by the user. It is content-based rather than collaborative filtering because the project does not use user ratings or watch histories.”

“I started by joining the movies and credits tables, then retained movie ID, title, overview, genres, keywords, cast and crew. The genres, keywords, cast and crew columns contained JSON-like strings, so I used ast.literal_eval to parse them. I extracted genre and keyword names, selected the first three cast members, and filtered the crew to directors. I also removed spaces within multi-word names so they would remain single tokens. Then I split the overview into words and combined all of these fields into one tags string for each movie.”

“Next, I used CountVectorizer with a maximum vocabulary of 5,000 features and English stop-word removal. This converts every movie into a bag-of-words vector. I calculated pairwise cosine similarity between the vectors. Cosine similarity compares the angle between two vectors, so it focuses more on their relative term composition than on raw document length. When a user selects a movie, the code retrieves its similarity scores, sorts all movies in descending order, skips the selected movie itself and returns the next five titles.”

“For example, selecting Gandhi returns Gandhi, My Father, The Wind That Shakes the Barley, A Passage to India, Guiana 1838 and Ramanujan. The repository demonstrates this qualitatively; it does not have user feedback or relevance labels, so I would not claim a measured precision or recall.”

“The main limitations are that exact word overlap misses semantic relationships, all metadata fields are combined without weights, and there is no personalization beyond the selected title. The full pairwise similarity matrix also grows quadratically. I would improve it with TF-IDF or embeddings, field weighting, duplicate-title handling, diversity re-ranking and evaluation using Precision at K and NDCG at K. If interaction data became available, I would consider a hybrid recommender.”

Likely interview questions

1. What is content-based filtering? Recommending items based on their own attributes and similarity to an item/profile the user likes.

2. How is it different from collaborative filtering? Collaborative filtering uses user-item interactions or rating patterns; this project uses movie metadata only.

3. Why CountVectorizer? It turns textual tags into fixed-length numeric count vectors that can be compared mathematically.

4. What is the bag-of-words limitation? It ignores word order and semantic similarity; synonyms may have no overlap.

5. Why cosine similarity? It compares vector direction and is less sensitive to document length than raw Euclidean distance.

6. Why remove spaces in cast/director names? To keep a multi-word name as a single token rather than separate common words.

7. Why take only three cast members? To focus on leading cast and limit noisy/long lists, though this can omit relevant actors.

8. What is cold start? New items can be recommended if metadata exists, even without interactions; sparse metadata still hurts.

9. How would you evaluate? Human relevance judgments or user interactions, with Precision@K, Recall@K, MAP@K, NDCG@K, coverage and diversity.

10. What is the scalability issue? The dense N×N similarity matrix has quadratic memory and computation requirements.

11. Why use TF-IDF? It downweights common terms and emphasizes more discriminative words.

12. Why is merge-on-title risky? Titles are not guaranteed unique or consistently formatted; use stable movie IDs and validate join cardinality.